AWS IAM Identity Center + Microsoft Entra SSO Federation

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — AWS IAM Identity Center + Microsoft Entra SSO Federation 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — AWS IAM Identity Center + Microsoft Entra SSO Federation 5

What this solution is . . . 5

How federation works . . . 5

The objects this solution creates (naming convention) . . . 6

The standard deployment model . . . 6

Scope . . . 6

What is out of scope . . . 7

Authentication and connections used . . . 7

2. Requirements — AWS IAM Identity Center + Microsoft Entra SSO Federation 7

R1. AWS Organizations enabled with all features . . . 8

R2. IAM Identity Center available in your chosen home Region . . . 8

R3. Microsoft Entra ID tenant with app + provisioning rights . . . 8

R4. AWS access portal URL and SAML/SCIM exchange readiness . . . 8

R5. A non-production ("sandbox") member account for the pilot . . . 9

R6. Defined access tiers and group-to-role mapping (customer decision) . . . 9

R7. Entra ID licensing for the provisioning/protection features you intend to use . . . 9

R8. Inventory of existing IAM users and access keys (for decommissioning) . . . 9

Global Requirements — AWS (category-level) 9

G1. AWS account(s) with management/root access to the org . . . 10

G2. Least-privilege deployment identity (no long-lived root) . . . 10

G3. AWS CLI and IaC tooling . . . 10

G4. Source control + CI/CD . . . 10

G5. Terraform remote state . . . 10

G6. Region and tagging standards . . . 10

G7. Identity provider for federation (where required) . . . 11

G8. Non-production account + change control . . . 11

3. Implementation — AWS IAM Identity Center + Microsoft Entra SSO Federation 11

Stage A — Assess . . . 11

Stage B — Configure . . . 12

Stage C — Pilot (sandbox + pilot group) . . . 14

Stage D — Validate . . . 14
Stage E — Expand and decommission . . . 14
5. Testing — AWS IAM Identity Center + Microsoft Entra SSO Federation 15
Gate 1 — SAML sign-in works . . . 15
Gate 2 — SCIM provisioning synced the right objects . . . 16
Gate 3 — Least-privilege boundary holds . . . 16
Gate 4 — Deprovisioning (offboarding) works . . . 16
Gate 5 — Session controls behave . . . 16
Gate 6 — Break-glass path is proven . . . 16
Gate 7 — Assignment matrix matches intent . . . 16
6. Troubleshooting — AWS IAM Identity Center + Microsoft Entra SSO Federation 17
T1. Sign-in fails with a SAML error ("invalid response", signature/audience mismatch) . . . 17
T2. Redirect loop or "you do not have access" after authenticating . . . 17
T3. Users or groups don't appear in AWS (SCIM provisioning) . . . 17
T4. A read-only user can make changes (least-privilege breach) . . . 17
T5. Break-glass sign-in fails or isn't alerted . . . 18
T6. Terraform plan wants to destroy/replace permission sets . . . 18
T7. Provisioning "Test Connection" fails immediately . . . 18
T8. Wrong or missing accounts in a user's portal after Expand . . . 18
7. Best Practices — AWS IAM Identity Center + Microsoft Entra SSO Federation 18
Keep it clean . . . 18
Defer the decisions that aren't yours . . . 19
Where to go next (adjacent catalog areas) . . . 19
8. Deliverables Summary — AWS IAM Identity Center + Microsoft Entra SSO Federation 20
Documentation . . . 20
Automation artifacts . . . 20
What the buyer receives . . . 21
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — AWS IAM Identity Center + Microsoft Entra SSO Federation

AWS

Standalone IAM users are the oldest and leakiest way to get into AWS: each is a static credential with its own password and access keys, created and deleted by hand, with no tie to your corporate identity and no automatic offboarding. When someone leaves, their AWS logins linger until a human remembers to remove them. This solution replaces that model with federated single sign-on: AWS IAM Identity Center is enabled org-wide, Microsoft Entra ID becomes the identity provider over SAML, users and groups are pushed into AWS automatically over SCIM, and Entra groups are mapped to least-privilege permission sets across every AWS account.

This is a complete, deploy-it-carefully package for federating AWS IAM Identity Center with Microsoft Entra ID, written for an administrator who knows the Microsoft side but is new to AWS Organizations and IAM Identity Center.

What you get

• A plain-language Solution Overview of how federation works — Entra ID (SAML IdP) to IAM Identity Center (service provider), SCIM for user/group provisioning, permission sets mapped to Entra groups, and account assignments across the organization.

• A Requirements Guide with verify / implement-if-missing for every solution-specific prerequisite (the AWS category prerequisites are published once in the separate Global Requirements).

• An Implementation Guide of precise, ordered steps on both consoles — enable IAM Identity Center and delegated admin, build the Entra enterprise app (SAML + SCIM), create named permission sets, and assign access — always in a sandbox account first.

• Five automation artifacts: an AWS CLI discovery script, an Entra-side validation script (Microsoft.Graph, read-only), an AWS CLI reporting script, a Terraform configuration for permission sets and assignments (plan-first), and a gated Entra provisioning helper (`-WhatIf` by default).

• Testing and Troubleshooting guides for a confident rollout, plus break-glass and legacy-IAM-user decommissioning.

What this delivers

One Microsoft identity signing in to every AWS account: Entra users authenticate once, land in the AWS access portal, and see only the accounts and roles their Entra group membership grants — least-privilege permission sets, auto-provisioned and auto-deprovisioned, with a tested emergency break-glass path and no standalone IAM users left to leak.

The standard deployment model

`Assess -> Configure -> Pilot -> Validate -> Expand`, with a pilot gate that validates SSO sign-in and SCIM provisioning against a sandbox account and a small pilot group before any production account is assigned or any legacy IAM user is disabled.

*IAM Identity Center feature availability, region support, and any premium Entra ID licensing (for example Entra ID P1/P2 for certain provisioning and Conditional Access capabilities) change over time — *verify current availability and licensing in your own tenant and AWS organization before you rely on a specific capability.**

Outcome

Corporate identities from Entra ID federated into AWS with least-privilege, automatically provisioned access across the organization — proven on a pilot, with break-glass in place, before legacy IAM users are retired.

1. Solution Overview — AWS IAM Identity Center + Microsoft Entra SSO Federation

What this solution is

AWS IAM Identity Center (formerly AWS SSO) is the AWS service that gives your workforce a single sign-on portal into every AWS account in your organization. Instead of each account holding its own IAM users and passwords, IAM Identity Center sits at the organization level and hands out short-lived, role-based access. This solution connects IAM Identity Center to Microsoft Entra ID as the external identity provider (IdP): Entra ID authenticates the user (SAML) and pushes user and group objects into AWS (SCIM), and you map each Entra group to a least-privilege permission set that applies to the AWS accounts you choose.

The result is that identity lives in one place — Entra ID. AWS trusts Entra to say who the user is, and AWS decides what that user can do based on their Entra group membership.

How federation works

1. **Authentication (SAML).** A user opens the AWS access portal URL. IAM Identity Center redirects them to Entra ID, which authenticates them (with your existing MFA and Conditional Access), then returns a signed SAML assertion. AWS trusts the assertion and signs the user in. No AWS password exists.

2. **Provisioning (SCIM).** Entra ID's provisioning service pushes the assigned users and groups into IAM Identity Center over SCIM on a schedule. When you add a user to an Entra group, that user appears in AWS automatically; when you remove them, they are deprovisioned automatically. This is what replaces manual IAM-user lifecycle.

3. **Authorization (permission sets + assignments).** A **permission set** is a named collection of AWS permissions (managed and/or inline policies plus a session duration) that IAM Identity Center materializes as an IAM role inside each account it is assigned to. An **account assignment** links *Entra group -> permission set -> AWS account*. Group membership therefore drives exactly which accounts and roles a person sees.

4. **Access.** In the portal the user sees a tile per account/role they are entitled to, clicks it, and gets a time-boxed console session or CLI credentials — never a static key.

The objects this solution creates (naming convention)

Every object follows `<Purpose> – <Scope> – <Stage>` so the same names appear in the steps, the scripts, the tests, and troubleshooting. The pilot set is:

- **Entra enterprise application:** `AWS IAM Identity Center – Federation – SSO` (the SAML + SCIM app).
- **Permission sets (materialize as IAM roles per account):**
- `SSO-Admin – Sandbox – Pilot` — administrative access, **sandbox/non-production account only**, for the pilot.
- `SSO-ReadOnly – AllAccounts – Prod` — read-only across assigned accounts.
- `SSO-PowerUser – Workloads – Prod` — power-user (no IAM/billing) on workload accounts.
- `SSO-Billing – Management – Prod` — billing/cost access scoped to the management account.
- `SSO-BreakGlass – Management – Emergency` — emergency administrative access, assigned to a tiny, monitored group (see break-glass).
- **Entra security groups (mapped 1:1 to the permission sets):** `AWS-SSO-Admin-Sandbox-Pilot`, `AWS-SSO-ReadOnly-AllAccounts`, `AWS-SSO-PowerUser-Workloads`, `AWS-SSO-Billing-Management`, `AWS-SSO-BreakGlass-Emergency`.
- **Delegated administration:** a designated member account (referred to as the **Identity account**) registered as the IAM Identity Center delegated administrator so day-to-day SSO administration does not require the management account.

The standard deployment model

> *Assess the current IAM-user footprint and org readiness, configure IAM Identity Center + the Entra app + permission sets, pilot SSO and SCIM against the sandbox account with one pilot group, validate sign-in/provisioning/least-privilege at explicit gates, then expand to production accounts and decommission legacy IAM users. Standard pattern:* `Assess -> Configure -> Pilot -> Validate -> Expand`.

- **Assess** — inventory IAM users, access keys, and existing roles; confirm Organizations, Entra rights, and licensing.
- **Configure** — enable IAM Identity Center, register delegated admin, build the Entra enterprise app (SAML + SCIM), create the named permission sets.
- **Pilot** — assign only `SSO-Admin – Sandbox – Pilot` and `SSO-ReadOnly – AllAccounts – Prod` to the pilot group against the sandbox account; prove sign-in and provisioning end to end.
- **Validate** — run the pass/fail gates in the Testing Guide (SAML sign-in, SCIM sync, least-privilege boundary, break-glass).
- **Expand** — assign production accounts group by group, then disable and remove legacy IAM users on a schedule with rollback.

Scope

• Enabling AWS IAM Identity Center at the organization level and registering a delegated administrator.

• Configuring Microsoft Entra ID as the SAML IdP and enabling SCIM user/group provisioning into IAM Identity Center.

• Creating least-privilege permission sets and mapping Entra groups to them across selected AWS accounts.

• Session controls (session duration, sign-in experience) and a tested break-glass path.

• A staged decommissioning plan for legacy standalone IAM users and their access keys.

What is out of scope

• Designing the AWS Organizations structure, OUs, and SCPs themselves (that is the AWS landing-zone solution — see *Where to go next*).

• Full Entra ID governance (PIM, access reviews, entitlement management) — this solution consumes Entra groups; it does not build the governance program around them.

• Application-level SSO to non-AWS SaaS, and customer-managed permission policies beyond the permission sets defined here.

• The customer's decision on *which* people belong in *which* access tier — that access-governance determination belongs to the customer's security and identity owners; this guide provides the technical mapping, not the entitlement policy.

Authentication and connections used

• **AWS side:** an assumed, least-privilege role or IAM Identity Center permission set in the management/Identity account with rights to Organizations and `sso-admin` / `identitystore` APIs — never root, never long-lived admin keys (see Global Requirements G1/G2).

• **Entra side:** an admin with rights to create and configure an enterprise application, assign users/groups, and configure provisioning (Application Administrator or higher). The provided PowerShell uses **Microsoft.Graph** with delegated, read-only scopes for validation and an explicitly gated scope for the optional provisioning helper.

• **Terraform:** the AWS provider using the assumed role above, run **plan-first** through source control against a non-production account before production (Global Requirements G3–G5, G8).

*Some capabilities referenced here (certain SCIM group-provisioning behaviors, Conditional Access, and identity-protection features) depend on your Entra ID plan and on current AWS region/feature support. *Verify current availability and licensing in your tenant and organization before relying on a specific capability.**

2. Requirements — AWS IAM Identity Center + Microsoft Entra SSO Federation

The AWS category prerequisites (management/root access, least-privilege deployment identity, AWS CLI + Terraform toolchain, source control + CI/CD with OIDC, remote state, region/tagging standards, an identity provider, and a non-production account with change control) are published once in **Global**

Requirements — AWS and are not repeated here. Confirm those first. Below are the prerequisites **specific to federating IAM Identity Center with Entra ID**. Each item: Description / Why required / How to verify / How to implement if missing.

R1. AWS Organizations enabled with all features

Description: The AWS account you run this from must be part of an organization created in AWS Organizations, with **all features** enabled (not just consolidated billing). Why required: IAM Identity Center is an organization-level service; org-wide enablement, delegated administration, and multi-account assignments require Organizations with all features. How to verify: In the AWS Organizations console the management account shows "All features" enabled; `aws organizations describe-organization` returns `"FeatureSet": "ALL"`. How to implement if missing: Enable all features in AWS Organizations (management account) and accept the invitation from member accounts, or deploy the AWS landing-zone solution first. *Enabling all features is a one-way action approved by the org owner.*

R2. IAM Identity Center available in your chosen home Region

Description: A single AWS Region designated as the IAM Identity Center "home" (identity store) Region, supported in your partition. Why required: IAM Identity Center has one instance per organization anchored to a home Region; the SAML/SCIM endpoints and identity store live there. How to verify: In the IAM Identity Center console you can enable the service and it reports a home Region; `aws sso-admin list-instances` returns an instance ARN. *Verify current IAM Identity Center Region/feature availability for your partition (commercial vs GovCloud differ).* How to implement if missing: Choose a supported home Region aligned to your data-residency standard (Global Requirements G6) and enable IAM Identity Center there.

R3. Microsoft Entra ID tenant with app + provisioning rights

Description: An Entra ID tenant and an admin who can create an enterprise application, configure SAML SSO, assign users/groups, and configure provisioning. Why required: Entra ID is the SAML IdP and the SCIM provisioning source; building the app and turning on provisioning both require these rights. How to verify: You can open **Entra admin center -> Enterprise applications -> New application** and, on an app, see the **Single sign-on** and **Provisioning** blades. Role is Application Administrator/Cloud Application Administrator or higher. How to implement if missing: Have an Entra Global/Application Administrator grant the role, or perform the app-creation steps. Confirm plan-dependent features (see R7).

R4. AWS access portal URL and SAML/SCIM exchange readiness

Description: Ability to exchange federation metadata both directions — download AWS's IAM Identity Center SAML metadata, and take the Entra SAML metadata / SCIM tenant URL + secret token back into AWS. Why required: SAML trust and SCIM provisioning each require configuration values to be copied between the two consoles in the correct order. How to verify: In IAM Identity Center **Settings -> Identity source** you can change the identity source to "External identity provider" and download the AWS SAML metadata; on the Entra app you can retrieve the SCIM **Tenant URL** and **Secret Token** fields. How to implement if missing: Nothing to pre-build — this readiness is confirmed during implementation. Ensure both admins are available in the same change window so values can be exchanged without stale tokens.

R5. A non-production ("sandbox") member account for the pilot

Description: A dedicated non-production AWS account in the organization to receive the pilot permission-set assignments. Why required: The pilot assigns `SSO-Admin – Sandbox – Pilot` here so administrative testing never touches production, satisfying the pilot-first gate. How to verify: The account exists in Organizations and is in a non-production OU; you can assign access to it from IAM Identity Center. How to implement if missing: Create or designate a sandbox account (Global Requirements G8) before the pilot.

R6. Defined access tiers and group-to-role mapping (customer decision)

Description: A documented decision on which access tiers exist and which people/groups belong in each (admin, power-user, read-only, billing, break-glass). Why required: Permission sets and assignments encode an access policy; the technical mapping is only as safe as the entitlement decision behind it. How to verify: A short RACI/entitlement matrix exists naming the Entra groups and the intended AWS scope for each. How to implement if missing: The customer's security/identity owners define the tiers and membership. *This entitlement determination is the customer's decision; this solution implements the mapping, not the policy.*

R7. Entra ID licensing for the provisioning/protection features you intend to use

Description: The Entra ID plan (Free/P1/P2) that covers the provisioning, Conditional Access, and identity-protection features you plan to apply to the AWS app. Why required: SCIM app provisioning and Conditional Access targeting of the AWS app depend on plan level; identity-protection/PIM features require premium plans. How to verify: In the Entra admin center check the tenant's licensing; confirm the specific feature (e.g. Conditional Access, group-based assignment) is available on your plan. *Verify current licensing — Entra plan names and feature gating change; premium features such as Conditional Access and Identity Protection typically require Entra ID P1/P2.* How to implement if missing: Acquire the appropriate Entra plan for the features you require, or scope the design to what your current plan supports.

R8. Inventory of existing IAM users and access keys (for decommissioning)

Description: A current inventory of standalone IAM users, their access keys, last-used dates, and what depends on them (scripts, CI, service accounts). Why required: Legacy IAM users are only disabled/removed after SSO is proven; you must know which are human logins (replaced by SSO) versus programmatic identities (which move to roles/OIDC, not SSO). How to verify: Run the provided `AWSSSO-Discovery.sh` (or `aws iam generate-credential-report`) to list users, keys, and last-used data. How to implement if missing: Generate the inventory before the Expand stage; classify each user as human (retire via SSO) or workload (migrate to a role/OIDC separately, out of scope here).

Once R1–R8 are confirmed alongside the AWS Global Requirements, proceed to the Implementation Guide. Everything is done in the sandbox account and a small pilot group first.

Global Requirements — AWS (category-level)

These requirements apply to every solution in the AWS category (foundation and security on Amazon Web Services). Published once at the category level so listings stay small. Confirm before deploying any AWS

solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *AWS changes create real, billable resources across accounts. Everything runs plan/what-if first through version control, in a non-production account before production, with least-privilege credentials — never long-lived root or admin keys on a laptop.*

G1. AWS account(s) with management/root access to the org

Description: An AWS management (payer) account, or Owner-equivalent rights to create Organizations resources. Why: Landing-zone, Organizations, SCP, and delegated-admin actions operate at the org/management scope. Verify: `aws sts get-caller-identity` succeeds; the identity can access AWS Organizations. Implement: Obtain the management account (or delegated rights) before org-level changes.

G2. Least-privilege deployment identity (no long-lived root)

Description: An IAM role / IAM Identity Center permission set (or OIDC-federated CI identity) with scoped rights — not root, not static admin keys. Why: Root and long-lived admin keys are the top AWS breach vector; automation should assume a scoped role. Verify: Deployments run as an assumed role; root MFA is on and root keys do not exist. Implement: Create a scoped deployment role / permission set; enable root MFA; delete any root access keys.

G3. AWS CLI and IaC tooling

Description: AWS CLI v2, Terraform CLI (pinned), and Git installed. Why: The solutions deliver Terraform + AWS CLI; version drift breaks reproducibility. Verify: `aws --version`, `terraform version`, `git --version` return supported builds. *Verify current provider/CLI versions.* Implement: Install and pin the toolchain; record versions in the repo.

G4. Source control + CI/CD

Description: A Git repository and a pipeline (GitHub Actions / GitLab CI / CodePipeline) with OIDC federation to AWS. Why: Plan/apply must be reviewed and run through an auditable pipeline, not locally. Verify: The pipeline can assume an AWS role via OIDC and run `terraform plan`. Implement: Configure the repo and OIDC trust; scope the CI role.

G5. Terraform remote state

Description: A remote state backend (S3 + DynamoDB lock, or Terraform Cloud). Why: Local state is unsafe and unshareable; remote state with locking prevents corruption. Verify: The state bucket/table (or TFC workspace) exists and is access-controlled and encrypted. Implement: Provision the S3 state bucket + DynamoDB lock table with encryption and versioning.

G6. Region and tagging standards

Description: Approved AWS regions, a tagging standard, and naming conventions. Why: Governance, cost allocation, and SCP/Config rules depend on consistent regions/tags. Verify: A documented standard exists and is encoded in modules/policy. Implement: Define the standard; enforce via tag policies and Config

rules.

G7. Identity provider for federation (where required)

Description: Microsoft Entra ID (or another IdP) available to federate AWS IAM Identity Center. Why: SSO/SCUM federation solutions need the IdP side configured. Verify: Entra admin access to create an enterprise app (SAML + SCIM). *Verify current IAM Identity Center feature availability.* Implement: Ensure Entra rights (see the catalog's Entra solutions) before federating.

G8. Non-production account + change control

Description: A non-production account/OU to validate against, and a production change-approval process. Why: Every change is validated in non-prod and approved before production apply. Verify: A non-prod account exists; the pipeline requires approval before prod stages. Implement: Provision a non-prod account and add manual approval gates. *Production change approval is the customer's operational decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — AWS IAM Identity Center + Microsoft Entra SSO Federation

Follow these steps in order. You work across two consoles — the **AWS Management Console** (as an assumed least-privilege role in the management or delegated Identity account, never root) and the **Microsoft Entra admin center** (`entra.microsoft.com`). Every object uses the `<Purpose> – <Scope> – <Stage>` convention from the Overview and is reused verbatim in Testing, Troubleshooting, and the scripts. **Do the pilot against the sandbox account and one pilot group before touching any production account or IAM user.**

> *Portal navigation paths and blade names change. Where a path has moved, use the console search bar for the service/feature name. Confirm IAM Identity Center feature and Region availability for your partition before you begin.*

Stage A — Assess

Step A1. Confirm the organization and identity

1. Sign in to the AWS Management Console as your assumed deployment role.
2. Open **AWS Organizations**. Confirm the banner shows **All features** enabled and note the **management account ID** and the **sandbox account ID** (from R5).
3. Run the discovery script `AWSSSO-Discovery.sh` (read-only). It reports whether an IAM Identity Center instance exists, lists existing IAM users and access keys, and lists any existing permission sets. Save the output — it is your baseline.

Step A2. Confirm Entra rights and licensing

1. Sign in to the **Entra admin center** (`entra.microsoft.com`) as Application Administrator or higher.

2. Confirm you can reach **Entra ID -> Enterprise applications** and that **New application** is available.

3. Confirm the tenant licensing covers the features you plan to use (R7). *Verify current licensing.*

Stage B — Configure

Step B1. Enable IAM Identity Center

1. In the management account, open **IAM Identity Center**.

2. Choose your home Region (R2) in the Region selector, then choose **Enable**.

3. When prompted for the deployment, choose **Enable with AWS Organizations**. Wait for the instance to be created.

4. On the dashboard, note the **AWS access portal URL** (e.g. `https://d-xxxxxxxxxx.awsapps.com/start`). You will need it for the Entra SAML config and for user testing.

Step B2. Register the delegated administrator (Identity account)

1. Still in the management account, open **IAM Identity Center -> Settings -> Management**.

2. Under **Delegated administrator**, choose **Register account** and select your designated **Identity account** (a member account, not the management account).

3. From now on, perform day-to-day SSO administration signed into the **Identity account** where possible. (Changing the identity source in B4 must still be done from the management account.)

Step B3. Create the Entra enterprise application

1. In the Entra admin center, go to **Entra ID -> Enterprise applications -> New application**.

2. In the gallery, search for the **AWS IAM Identity Center** application and choose it. On the create pane set the name to `AWS IAM Identity Center – Federation – SSO` and choose **Create**.

3. This single gallery app supports both SAML SSO and SCIM provisioning, which you configure next.

Step B4. Establish SAML trust (both directions)

1. **AWS side (management account):** In **IAM Identity Center -> Settings -> Identity source**, choose **Actions -> Change identity source -> External identity provider**.

2. On that page, **download the AWS SAML metadata** file (the service-provider metadata). Leave this page open.

3. **Entra side:** open the `AWS IAM Identity Center – Federation – SSO` app -> **Single sign-on -> SAML**.

• In **Basic SAML Configuration**, upload the AWS metadata file (choose **Upload metadata file**) so the Identifier (Entity ID) and Reply URL populate from AWS. Save.

• In the **SAML Certificates** section, download the **Federation Metadata XML** (the Entra IdP metadata).

1. **Back on the AWS side:** upload the Entra **Federation Metadata XML** as the IdP SAML metadata, then choose **Next / Change identity source** to confirm. AWS now trusts Entra for authentication.

2. In Entra, on the app **-> Users and groups**, add yourself (a test admin) so you can sign in during the pilot.

Step B5. Enable SCIM provisioning (both directions)

1. **AWS side:** In **IAM Identity Center -> Settings -> Identity source**, find **Automatic provisioning** and choose **Enable**. AWS displays a **SCIM endpoint (Tenant URL)** and generates an **Access token (Secret Token)**. Copy both now — the token is shown once.

2. **Entra side:** on the `AWS IAM Identity Center – Federation – SSO` app -> **Provisioning -> Get started**, set **Provisioning Mode = Automatic**.

• Under **Admin Credentials**, paste the AWS **Tenant URL** and **Secret Token**, then choose **Test Connection**. It must succeed before saving.

• Save. Leave **Provisioning Status = Off** for now; you will turn it on in the pilot after scoping.

Step B6. Create the Entra security groups

1. In **Entra ID -> Groups -> New group**, create these security groups (assigned membership):

• `AWS-SSO-Admin-Sandbox-Pilot`

• `AWS-SSO-ReadOnly-AllAccounts`

• `AWS-SSO-PowerUser-Workloads`

• `AWS-SSO-Billing-Management`

• `AWS-SSO-BreakGlass-Emergency`

1. Add your pilot users only to `AWS-SSO-Admin-Sandbox-Pilot` and `AWS-SSO-ReadOnly-AllAccounts` for now. Populate the others during Expand.

2. On the app **-> Users and groups**, **Add** each of the five groups so they are in provisioning scope. (In **Provisioning -> Settings**, set **Scope = Sync only assigned users and groups**.)

Step B7. Create the permission sets (Terraform, plan-first)

1. Use the provided Terraform configuration `AWSSSO-permission-sets.tf` (AWS provider, `aws_ssoadmin_permission_set` + `aws_ssoadmin_managed_policy_attachment` + `aws_ssoadmin_account_assignment`). Set the input variables for your instance ARN, sandbox account ID, and Entra-provisioned group display names.

2. Run **plan-first** through your pipeline/remote state (Global Requirements G4/G5): `terraform plan`. Review that it will create exactly these permission sets:

• `SSO-Admin – Sandbox – Pilot` (AdministratorAccess managed policy; short session duration, e.g. PT1H).

• `SSO-ReadOnly – AllAccounts – Prod` (ReadOnlyAccess).

• `SSO-PowerUser – Workloads – Prod` (PowerUserAccess).

• `SSO-Billing – Management – Prod` (Billing + read-only, scoped to management account).

• `SSO-BreakGlass – Management – Emergency` (AdministratorAccess; assigned only to `AWS-SSO-BreakGlass-Emergency`).

1. **Apply only the sandbox-scoped resources first** (target the pilot assignment) after approval. Production account assignments are applied in Expand.

*If you prefer the console for the pilot: **IAM Identity Center -> Permission sets -> Create permission set** creates the same objects; name them exactly as above. Terraform remains the source of truth for repeatability.*

Stage C — Pilot (sandbox + pilot group)

Step C1. Turn on provisioning and confirm the sync

1. In Entra, on the app **-> Provisioning**, set **Provisioning Status = On** and save.

2. Wait for the initial cycle (it can take time). In **Provisioning -> View provisioning logs**, confirm your pilot users and the five groups provisioned successfully.

3. In AWS **IAM Identity Center -> Users** and **-> Groups**, confirm the same users/groups now appear (pushed by SCIM — you did not create them by hand).

Step C2. Assign pilot access (sandbox only)

1. Confirm the Terraform-applied (or console-created) assignments for the **sandbox account**:

• `AWS-SSO-Admin-Sandbox-Pilot` -> `SSO-Admin – Sandbox – Pilot` on the **sandbox account** only.

• `AWS-SSO-ReadOnly-AllAccounts` -> `SSO-ReadOnly – AllAccounts – Prod` on the **sandbox account** only for now.

1. Do **not** assign any production account yet.

Step C3. First federated sign-in

1. Open the **AWS access portal URL** (from B1) in a clean browser session.

2. Sign in — you are redirected to Entra, authenticate with MFA, and return to the portal.

3. Confirm you see tiles only for the sandbox account with the roles your group membership grants (Admin and ReadOnly). Click a tile and confirm you get a console session.

Proceed to the Testing Guide and pass every gate before Expand.

Stage D — Validate

Run the Testing Guide (`05-testing.md`) end to end: SAML sign-in, SCIM provisioning and **de**provisioning, the least-privilege boundary (ReadOnly cannot change anything), session duration, and the break-glass path. Record pass/fail. Do not expand until all gates pass.

Stage E — Expand and decommission

Step E1. Configure break-glass before production expansion

1. Keep **one** emergency IAM user in the **management account** named `breakglass-emergency` with a strong password, hardware MFA, and **no access keys**, stored offline (this is your recovery path if federation breaks — see Best Practices). Do not delete it during decommissioning.

2. Assign `AWS-SSO-BreakGlass-Emergency` -> `SSO-BreakGlass – Management – Emergency` to the management account, membership limited to one or two trusted admins, and enable CloudTrail alerting on its use.

Step E2. Assign production accounts (group by group)

1. Populate the Entra groups with their real members (per the R6 entitlement matrix).

2. Using `AWSSSO-permission-sets.tf`, add and **plan-first** the production account assignments:

• `AWS-SSO-ReadOnly-AllAccounts` -> `SSO-ReadOnly – AllAccounts – Prod` across the accounts they should read.

• `AWS-SSO-PowerUser-Workloads` -> `SSO-PowerUser – Workloads – Prod` on workload accounts.

• `AWS-SSO-Billing-Management` -> `SSO-Billing – Management – Prod` on the management account.

1. Apply after approval, one wave of accounts at a time. Re-run `AWSSSO-Report.sh` after each wave to confirm the assignment matrix matches the intended mapping.

Step E3. Decommission legacy IAM users (staged, with rollback)

1. From the R8 inventory, confirm which IAM users are **human logins** now replaced by SSO (retire) versus **workload identities** (migrate to roles/OIDC separately — out of scope).

2. For each human user, in this order: **deactivate access keys -> disable console access (remove login profile) -> wait an agreed soak period -> delete the user**. Deactivating first is reversible; deletion is last.

3. Keep the `breakglass-emergency` user. Re-run `AWSSSO-Discovery.sh` after each wave to confirm the shrinking IAM-user footprint and that no unexpected key is still in use.

Production change approval for disabling/removing IAM users is the customer's operational decision — schedule each wave through your change process with a rollback (re-enable the login profile) ready.

5. Testing — AWS IAM Identity Center + Microsoft Entra SSO Federation

Run every gate below against the **sandbox account** and **pilot group** before you expand to production or decommission any IAM user. Each gate has an explicit pass/fail. Reference the automation scripts by name where noted.

Gate 1 — SAML sign-in works

Procedure: Open the **AWS access portal URL** in a clean browser. Sign in with a member of `AWS-SSO-Admin-Sandbox-Pilot`. Pass: You are redirected to Entra, complete MFA, return to the portal, and see account tiles. Fail: You get a SAML error, a redirect loop, or "no access." See

Troubleshooting T1/T2.

Gate 2 — SCIM provisioning synced the right objects

Procedure: In Entra, open the app **-> Provisioning -> View provisioning logs**. In AWS, open **IAM Identity Center -> Users** and **-> Groups**. Optionally run `AWSSSO-Validate.ps1` (read-only) against the Entra app. Pass: The pilot users and all five `AWS-SSO-*` groups show **provisioned/success** in Entra and **appear in AWS** without being created by hand; `AWSSSO-Validate.ps1` reports provisioning **On**, connection healthy, and scope = assigned only. Fail: Users/groups missing in AWS, provisioning errors, or the connection test fails. See Troubleshooting T3.

Gate 3 — Least-privilege boundary holds

Procedure: Sign in as a user who is **only** in `AWS-SSO-ReadOnly-AllAccounts`. Open the sandbox account with the `SSO-ReadOnly – AllAccounts – Prod` role. Attempt a change (e.g. create an S3 bucket or stop an instance). Pass: You can view resources but every write/change is **denied**; the user sees no Admin tile. Fail: A read-only user can make changes, or sees roles they should not. Re-check the permission-set policy and assignments (`AWSSSO-Report.sh`). See Troubleshooting T4.

Gate 4 — Deprovisioning (offboarding) works

Procedure: Remove a pilot user from `AWS-SSO-Admin-Sandbox-Pilot` (and any other AWS group) in Entra. Wait for the next provisioning cycle. Have that user try the portal. Pass: Within the sync window the user loses the corresponding tiles/roles; provisioning logs show the update. This proves lifecycle automation replaces manual IAM-user offboarding. Fail: The user retains access after removal. Check provisioning status/scope and sync latency. See Troubleshooting T3.

Gate 5 — Session controls behave

Procedure: Sign into a role and note the session duration configured on the permission set (e.g. PT1H for `SSO-Admin – Sandbox – Pilot`). Stay idle past expiry or check the CLI credential expiry. Pass: The session expires at the configured duration and re-authentication is required; no static keys were ever issued. Fail: Sessions do not expire as configured. Re-check the permission-set session duration.

Gate 6 — Break-glass path is proven

Procedure: In a controlled test, sign in as the `breakglass-emergency` IAM user (management account) with its MFA, and separately confirm a member of `AWS-SSO-BreakGlass-Emergency` can reach `SSO-BreakGlass – Management – Emergency`. Confirm CloudTrail/alerting fires on the emergency sign-in. Pass: Both emergency paths work and are alerted on. You have a recovery route if federation ever breaks. Fail: Break-glass sign-in fails or is not alerted. Fix before you disable any legacy IAM user (Troubleshooting T5).

Gate 7 — Assignment matrix matches intent

Procedure: Run `AWSSSO-Report.sh` (read-only). Compare the *group -> permission set -> account* matrix to the R6 entitlement decision. Pass: Every assignment is intended; no account/role is granted that the entitlement matrix does not list. Fail: Extra or missing assignments. Correct via

`AWSSSO-permission-sets.tf` (plan-first) and re-run.

When Gates 1–7 pass in the sandbox with the pilot group, proceed to Expand (production assignments, then staged IAM-user decommissioning). Re-run Gate 3 and Gate 7 after each production wave.

6. Troubleshooting — AWS IAM Identity Center + Microsoft Entra SSO Federation

Symptom -> cause -> fix for the failures you are most likely to hit. Work in the sandbox account with the pilot group; never troubleshoot by widening permissions in production.

T1. Sign-in fails with a SAML error ("invalid response", signature/audience mismatch)

Symptom: After Entra authentication the AWS portal shows a SAML error instead of account tiles. Cause: The SAML trust metadata is stale or one-directional — AWS and Entra are not using each other's current metadata, or the certificate rotated. Fix: Re-run **Step B4**. Re-download AWS's service-provider metadata into Entra's Basic SAML Configuration, and re-upload Entra's current **Federation Metadata XML** into the AWS external IdP configuration. Confirm the Entra SAML signing certificate is active (not expired) and re-test.

T2. Redirect loop or "you do not have access" after authenticating

Symptom: Sign-in succeeds at Entra but bounces back, or shows no accessible accounts. Cause: The user is authenticated but not **assigned** to the app and/or not a member of any group that has an account assignment. Fix: In Entra, on `AWS IAM Identity Center – Federation – SSO` **-> Users and groups**, confirm the user (or their group) is added. In AWS, confirm an **account assignment** exists linking that group to a permission set on an account (`AWSSSO-Report.sh`). Authentication (SAML) and authorization (assignment) are separate — both are required.

T3. Users or groups don't appear in AWS (SCIM provisioning)

Symptom: A pilot user/group is in Entra but never shows in **IAM Identity Center -> Users/Groups**, or an offboarded user keeps access. Cause: Provisioning is Off, the SCIM token/endpoint is wrong, the object is out of scope, or you are inside the sync window. Fix: In Entra **-> Provisioning**: confirm **Status = On** and **Scope = Sync only assigned users and groups**, and that the user/group is under **Users and groups**. Run **Test Connection** — if it fails, the AWS SCIM **Secret Token** likely expired or was mistyped; regenerate it in **IAM Identity Center -> Settings -> Automatic provisioning** and paste the new token. Check **View provisioning logs** for per-object errors. For deprovisioning, remember it follows the cycle — for urgent offboarding, also disable the user in Entra (`AWSSSO-Validate.ps1` reports provisioning state).

T4. A read-only user can make changes (least-privilege breach)

Symptom: A member of `AWS-SSO-ReadOnly-AllAccounts` can modify resources, or sees an Admin tile. Cause: The wrong managed policy is attached to the permission set, or the user is also in a higher-privilege group, or a stale console-created assignment exists. Fix: Inspect the permission set

(`SSO-ReadOnly – AllAccounts – Prod`) — it must attach `ReadOnlyAccess` only. Confirm the user's Entra group membership; a user in both ReadOnly and PowerUser groups gets the union. Reconcile against `AWSSSO-permission-sets.tf` (plan-first) so Terraform is authoritative, and re-run Gate 3.

T5. Break-glass sign-in fails or isn't alerted

Symptom: The `breakglass-emergency` IAM user cannot sign in, or its use raises no alert. Cause: The emergency user's MFA/login profile is misconfigured, or CloudTrail alerting on it was never wired. Fix: Confirm the `breakglass-emergency` user has a valid password and registered MFA, and **no access keys**. Confirm a CloudTrail-based alert (e.g. EventBridge rule on that principal) notifies your team on sign-in. **Never** disable a legacy IAM user (Stage E3) until Gate 6 passes — break-glass is your recovery path.

T6. Terraform plan wants to destroy/replace permission sets

Symptom: `terraform plan` shows it will replace or delete a permission set you created in the console. Cause: The console-created object and the Terraform-declared object are not the same resource — Terraform doesn't know about the console object (no state), so it plans to create its own and may collide. Fix: Import the existing permission set into Terraform state (`terraform import` on `aws_ssoadmin_permission_set`) rather than letting it recreate, or delete the console object and let Terraform own it. Standardize on Terraform going forward (Best Practices). Always review the plan before apply.

T7. Provisioning "Test Connection" fails immediately

Symptom: In Entra the SCIM Test Connection fails before you can save credentials. Cause: The AWS **Tenant URL** or **Secret Token** is wrong/expired, or automatic provisioning was never enabled on the AWS side. Fix: In **IAM Identity Center -> Settings**, confirm **Automatic provisioning** is enabled and copy a freshly generated **Access token** and the exact **SCIM endpoint** URL. Paste both into Entra Admin Credentials and re-test. The token is shown once on generation — regenerate if you lost it.

T8. Wrong or missing accounts in a user's portal after Expand

Symptom: After a production wave, users see too many or too few accounts. Cause: Account assignments were applied to the wrong accounts, or group membership doesn't match the entitlement matrix. Fix: Run `AWSSSO-Report.sh` and diff the *group -> permission set -> account* matrix against the R6 entitlement decision. Correct assignments in `AWSSSO-permission-sets.tf` (plan-first) and correct Entra group membership. Re-run Gate 7.

7. Best Practices — AWS IAM Identity Center + Microsoft Entra SSO Federation

Keep it clean

• **One identity source, one truth.** Once Entra ID is the identity source, create and manage all human access through Entra groups — never hand-create users inside IAM Identity Center. Group membership in Entra is the single control point.

• **Least privilege by default.** Start from `SSO-ReadOnly – AllAccounts – Prod` and grant up only where a role genuinely needs it. Reserve `AdministratorAccess` (the Admin and BreakGlass sets) for the smallest possible groups and the shortest session durations.

• **Sandbox before production, always.** The pilot uses `SSO-Admin – Sandbox – Pilot` on a non-production account precisely so admin testing never risks production. Keep that separation permanently — do not repurpose the sandbox set for prod.

• **Terraform is the source of truth.** Make permission sets and account assignments through `AWSSSO-permission-sets.tf`, plan-first, reviewed in a pull request. Console clicks drift; version-controlled assignments are auditable and reproducible.

• **Never delete break-glass.** Keep exactly one `breakglass-emergency` IAM user in the management account with hardware MFA, no access keys, credentials stored offline, and CloudTrail alerting on its use. It is your only way back in if federation breaks. Test it on a schedule (Gate 6).

• **Scope SCIM to assigned only.** Keep provisioning scope = "Sync only assigned users and groups" so you never accidentally push the whole directory into AWS.

• **Watch the SCIM sync window.** Deprovisioning is not instant — it follows the provisioning cycle. For immediate offboarding of a high-risk departure, disable the user in Entra (blocks sign-in immediately) as well as removing group membership.

• **Session duration is a control.** Short durations for privileged sets reduce the blast radius of a stolen session. Tune per tier, not globally.

• **Decommission legacy IAM users in the safe order.** Deactivate keys -> remove login profile -> soak -> delete. Deactivation is reversible; deletion is last. Keep re-runs of `AWSSSO-Discovery.sh` as evidence the footprint is shrinking.

• **Separate humans from workloads.** SSO is for people. Programmatic identities (CI, apps, service accounts) belong on IAM roles / OIDC federation, not IAM Identity Center — migrate them on a separate track.

• **Re-verify feature and licensing facts.** IAM Identity Center Region/feature support and Entra plan gating change. Before relying on Conditional Access, identity protection, or a specific provisioning behavior, *verify current availability in your tenant and organization.*

Defer the decisions that aren't yours

Who belongs in which access tier, how long privileged sessions may last under your policy, and when a legacy IAM user may be deleted are governance and change-control decisions for the customer's security, identity, and operations owners. This solution provides the technical controls and a safe rollout method — not the entitlement policy or the production change approval.

Where to go next (adjacent catalog areas)

• **AWS landing zone / Organizations baseline** — build or harden the OU structure, SCPs, and account factory that this SSO layer assigns access across.

• **AWS Security Hub & GuardDuty enablement** — turn on org-wide threat detection and posture scoring so the newly federated access is monitored.

• **Entra ID governance (PIM & access reviews)** — add just-in-time elevation and periodic recertification on top of the Entra groups that now drive AWS access.

• **Passwordless / phishing-resistant MFA (Entra)** — strengthen the authentication that AWS now trusts by federation.

• **Enterprise app & service principal governance (Entra)** — govern the `AWS IAM Identity Center – Federation – SSO` app itself alongside your other enterprise applications.

8. Deliverables Summary — AWS IAM Identity Center + Microsoft Entra SSO Federation

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How federation works (SAML auth, SCIM provisioning, permission sets + assignments), the named objects, scope, and the deployment model
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (AWS)	Category-wide AWS prerequisites, published once
3. Implementation Guide	Exact, ordered steps across the AWS and Entra consoles — enable IAM Identity Center, delegated admin, SAML + SCIM, permission sets, pilot, expand, decommission
5. Testing Guide	Seven pass/fail validation gates (sign-in, provisioning, least-privilege, deprovisioning, sessions, break-glass, assignment matrix)
6. Troubleshooting Guide	Symptom -> cause -> fix for the common federation failures
7. Best Practices	Keep-it-clean guidance and where to go next

Automation artifacts

Script	What it does
`AWSSSO-Discovery.sh`	AWS CLI, read-only. Reports whether IAM Identity Center is enabled, lists standalone IAM users + access keys (the decommission baseline), and lists existing permission sets/assignments

Script	What it does
`AWSSSO-Validate.ps1`	PowerShell (Microsoft.Graph), read-only. Validates the `AWS IAM Identity Center – Federation – SSO` enterprise app: SAML config present, provisioning On, connection healthy, scope = assigned only, and the five `AWS-SSO-*` groups in scope
`AWSSSO-Report.sh`	AWS CLI, read-only. Produces the *group -> permission set -> account* assignment matrix for comparison against the entitlement decision
`AWSSSO-permission-sets.tf`	Terraform (AWS provider). Defines the five named permission sets, managed-policy attachments, and account assignments. Run **plan-first**; apply sandbox scope for the pilot, production waves after approval
`AWSSSO-Entra-Provisioning.ps1`	PowerShell (Microsoft.Graph). Optional, **gated helper** that can create the enterprise app and the five Entra groups. Defaults to `-WhatIf` **(preview only)**; makes changes only when explicitly confirmed

All apply/change artifacts are plan/what-if-first and pilot-first: Terraform runs `plan` before `apply`, the provisioning helper previews by default, and every discovery/validation/report script is read-only.

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five automation artifacts (AWS CLI `.sh`, Terraform `.tf`, PowerShell `.ps1`) in a single ZIP — delivered as a Complete Solution Package (PDF + Word + scripts).
- A repeatable process following the standard model: `Assess -> Configure -> Pilot -> Validate -> Expand`.

Outcome

Corporate Entra ID identities federated into AWS IAM Identity Center with least-privilege, auto-provisioned access across the organization — proven on a sandbox pilot with break-glass in place before production expansion and legacy IAM-user decommissioning.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AWSSSO-Discovery.sh

Purpose: Reports the starting state before you federate: whether IAM Identity Center is enabled, the standalone IAM users and access keys that federation will replace (the decommission baseline), and any existing permission sets and account assignments. Read-only — it creates and changes nothing. HOW TO RUN Authenticate first as your least-privilege deployment role (never root): aws sts get-caller-identity # confirm the assumed role Then: ./AWSSSO-Discovery.sh # uses the current AWS CLI profile/region Requires: AWS CLI v2, jq. Rights: read on organizations, sso-admin, identitystore, and iam. EXPECTED RESULTS Console output (and a saved report file) showing: - Organization feature set and account IDs - IAM Identity Center instance ARN + identity store (or "not enabled") - Standalone IAM users, their access keys, and last-used dates - Existing permission sets and account assignments Use the IAM-user list as the R8 inventory for staged decommissioning. NOTES White-label. Read-only. Safe to run repeatedly (e.g. after each Expand wave) to watch the IAM-user footprint shrink. ========== set -euo pipefail

How to run: Authenticate first as your least-privilege deployment role (never root): aws sts get-caller-identity # confirm the assumed role Then: ./AWSSSO-Discovery.sh # uses the current AWS CLI profile/region Requires: AWS CLI v2, jq. Rights: read on organizations, sso-admin, identitystore, and iam. EXPECTED RESULTS Console output (and a saved report file) showing: - Organization feature set and account IDs - IAM Identity Center instance ARN + identity store (or "not enabled") - Standalone IAM users, their access keys, and last-used dates - Existing permission sets and account assignments Use the IAM-user list as the R8 inventory for staged decommissioning. NOTES White-label. Read-only. Safe to run repeatedly (e.g. after each Expand wave) to watch the IAM-user footprint shrink. ========== set -euo pipefail

AWSSSO-Entra-Provisioning.ps1

Purpose: Optional convenience for the Configure stage: creates the five Entra security groups (and, if requested, instantiates the AWS IAM Identity Center gallery application) so you do not click each one by hand. This is the ONLY script in the package that can write to Entra, so it is gated: it runs in preview (-WhatIf) mode unless you pass -Apply, and it never touches SAML certificates, SCIM tokens, or membership.

How to run: Install Graph once: Install-Module Microsoft.Graph -Scope CurrentUser Preview (default — makes NO changes): ./AWSSSO-Entra-Provisioning.ps1 Apply (creates the groups after your explicit confirmation): ./AWSSSO-Entra-Provisioning.ps1 -Apply Also create the enterprise app instance from the gallery: ./AWSSSO-Entra-Provisioning.ps1 -Apply -CreateApp

AWSSSO-Report.sh

Purpose: Produces the authoritative view of who can reach what: every account assignment, resolved to (Entra-provisioned group) -> (permission set) -> (AWS account). Compare this matrix against the R6 entitlement decision so no account/role is granted that the entitlement policy does not list. Read-only —

creates and changes nothing. HOW TO RUN Authenticate as your least-privilege role (never root), then: ./AWSSSO-Report.sh Requires: AWS CLI v2, jq. Rights: read on sso-admin, identitystore, organizations. EXPECTED RESULTS A printed and saved matrix, one row per assignment: ACCOUNT_ID | ACCOUNT_NAME | PERMISSION_SET | PRINCIPAL_TYPE | PRINCIPAL Run after each Expand wave (Testing Gate 7) to confirm intent. NOTES White-label. Read-only. Safe to run repeatedly. ========== set -euo pipefail

How to run: Authenticate as your least-privilege role (never root), then: ./AWSSSO-Report.sh Requires: AWS CLI v2, jq. Rights: read on sso-admin, identitystore, organizations. EXPECTED RESULTS A printed and saved matrix, one row per assignment: ACCOUNT_ID | ACCOUNT_NAME | PERMISSION_SET | PRINCIPAL_TYPE | PRINCIPAL Run after each Expand wave (Testing Gate 7) to confirm intent. NOTES White-label. Read-only. Safe to run repeatedly. ========== set -euo pipefail

AWSSSO-Validate.ps1

Purpose: Confirms the Entra enterprise application "AWS IAM Identity Center – Federation – SSO" is configured correctly for federation: the app exists, SAML single sign-on is set, provisioning (SCIM) is enabled with a healthy scope, and the five AWS-SSO-* security groups are present and in provisioning scope. Read-only — it reads Graph, changes nothing.

How to run: Install the Microsoft Graph module once: Install-Module Microsoft.Graph -Scope CurrentUser Then run (read-only delegated scopes): ./AWSSSO-Validate.ps1 You will be prompted to sign in to Entra. Consent to the read scopes: Application.Read.All, Directory.Read.All, Synchronization.Read.All.

AWSSSO-permission-sets.tf

Purpose: Declares the five named permission sets and the group -> permission set -> account assignments as code, so access is auditable, reviewable, and reproducible. Groups are provisioned into IAM Identity Center by Entra SCIM first; this config looks them up by display name and assigns access. HOW TO RUN (PLAN-FIRST, always) 1. Configure the AWS provider to ASSUME your least-privilege role (never root, never static admin keys) and a remote backend (S3 + DynamoDB lock). 2. terraform init 3. terraform plan -var-file=pilot.tfvars # REVIEW before applying 4. terraform apply -var-file=pilot.tfvars # pilot: sandbox scope only (Add production account IDs in a later var-file and re-plan/apply per Expand wave, after change approval.) EXPECTED RESULTS Terraform creates: SSO-Admin – Sandbox – Pilot, SSO-ReadOnly – AllAccounts – Prod, SSO-PowerUser – Workloads – Prod, SSO-Billing – Management – Prod, SSO-BreakGlass – Management – Emergency, and the assignments you list. NOTES White-label. Plan-first / pilot-first. Verify current AWS provider version and IAM Identity Center feature availability for your partition.
==========